# FastDedup

## A fast and memory efficient tool for reads deduplication

**Raphaël Ribes**
Master 2 Bioinformatics - Université de Montpellier

**Céline Mandier**
Study Engineer - Institut de Science des données de Montpellier

12 March 2026

**Abstract** – TODO

# Table of contents

1 – Context . . . 3
2 – Existing tools and limitations . . . 4
2.1 : Fastp . . . 4
2.2 : FastUniq . . . 4
2.3 : Clumpify . . . 4
2.4 : SeqKit . . . 5
2.5 : CD-HIT-DUP . . . 5
2.6 : PRINSEQ++ . . . 6
2.7 : SAM/BAM ecosystem tools . . . 6
3 – FastDedup: High-Performance FASTX Deduplication . . . 7
3.1 : Algorithmic Complexity and Hardware-Level Optimizations . . . 7
3.2 : The Mathematics of Dynamic Hashing . . . 8
3.3 : Architecture . . . 9
4 Benchmarking and performance evaluation . . . 10
5 Discussion and future perspectives . . . 12
6 Conclusion . . . 12
7 Acknowledgements . . . 12
Bibliography . . . 12

# 1 – Context

With the rise of New Generation Sequencing (NGS) technologies, the volume of sequencing data has increased exponentially. With this surge in data, the development of new tools for processing and analyzing sequencing reads was needed. One of the first and critical step in the analysis pipeline is the deduplication of reads, which involves identifying and removing duplicate sequences that can arise from PCR amplification or sequencing errors.

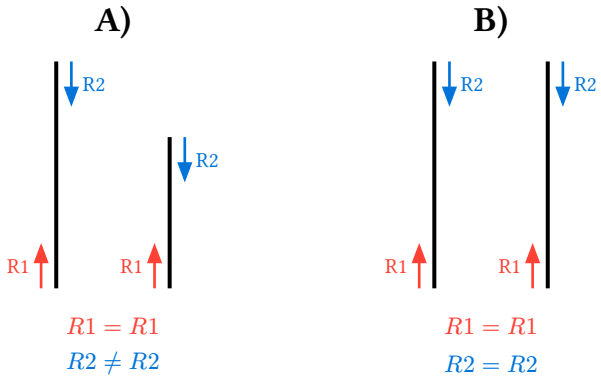


Figure 1: Identifying PCR Duplicates in Paired-End Sequencing.

**Solid Black Line:** Represents a single sequenced DNA fragment (height reflects length).

**Red Arrow (R1):** Forward read (Read 1).

**Blue Arrow (R2):** Reverse read (Read 2).

**A) Not a duplication:** Shows two distinct fragments where the start position of Read 1 is shared, but different fragment lengths lead to unique mapping positions for Read 2.

**B) PCR Duplication:** Shows two exact fragments where the start position of Read 1 and Read 2 is shared.

We define a fragment $F$, the 5′->3′ sequence as $R1$ with a length of $r_1$ and the 3′->5′ sequence as $R2$ with a length of $r_2$. We can say that $R1$ is the prefix of $F$, $R2$ reversed is the suffix of $F$ and $r_1 = r_2$.

Therefore, a PCR duplicate can be defined as if $F_1$ and $F_2$ have the same starting and ending position in the genome, with the same length. In that case, $R1_1$ and $R1_2$ are the same, and $R2_1$ and $R2_2$ are the same.

This step is crucial because removing these artifacts preventing false-positive during variant calling since PCR polymerases can introduce errors during the cycles of amplification. If an error occurs early, it is propagated into multiple daughter molecules. When sequenced, these identical, error-carrying molecules will appear as multiple independent reads supporting a mutation that does not actually exist in the biological sample [1].

Deduplication also allows correcting the allele frequency bias since not all DNA molecules amplify with equal efficiency due to factors like GC content or fragment length. This uneven amplification can artificially inflate the coverage of certain alleles. And it is also improving *de novo* assembly, especially in whole-genome sequencing. Paired-end reads are mapped to estimate the order and intervening distance between contigs, duplicates can create false-positive connections between contigs or introduce conflicting connections. [2].

Before presenting our tool, we will first review the existing tools for reads deduplication and their limitations.

# 2 – Existing tools and limitations

## 2.1 : Fastp

`fastp` is a widely adopted, all-in-one FASTQ preprocessor written in C++ that performs quality control, adapter trimming, and deduplication within a single pass [3,4]. Unlike traditional alignment-based tools that require computationally expensive mapping and coordinate sorting, `fastp` operates directly on raw sequences.

To achieve high performance, `fastp` implements a probabilistic deduplication mechanism based on Bloom filters and multiple hash arrays. For each read (or read pair), the sequence is mapped to multiple integer hash values. If all corresponding positions in the hash arrays are already marked as positive, the read is flagged as a duplicate. This streaming, single-pass architecture avoids reading and writing intermediate files to disk, drastically reducing I/O bottlenecks and resulting in ultra-fast execution speeds.

A significant advantage of `fastp` is its highly configurable memory management. Users can adjust the `--dup_calc_accuracy` parameter (ranging from level 1 to 6) to scale the hash buffer size from 1 GB up to 24 GB, allowing the tool to run on both standard personal computers and High-performance Computing (HPC) clusters.

However, this probabilistic approach involves a minor trade-off in absolute precision. `fastp` has a theoretical false-positive (hash collision) rate of approximately 0.01%, which, while negligible for most routine downstream analyses, represents a conceptual difference from exact deterministic methods. Furthermore, `fastp`'s deduplication algorithm relies heavily on the exact matching of sequence lengths and coordination regions. Consequently, applying certain quality trimming operations (such as sliding-window trimming via `--cut_front` or `--cut_tail`) prior to deduplication can interfere with the duplicate identification process, as trimmed reads may no longer perfectly match their duplicate counterparts.

## 2.2 : FastUniq

`FastUniq` [5] is a deterministic deduplication tool explicitly designed for paired-end (PE) short reads. By directly comparing the nucleotide sequences of read pairs, it provides an exact identification of PCR duplicates.

Algorithmically, `FastUniq` operates through a strict three-step pipeline:
1. loads the entirety of the paired reads into memory
2. sorts them based on their sequences using a merge sort algorithm
3. identifies duplicates by sequentially comparing adjacent read pairs in the sorted list.

A notable feature of its comparison logic is its ability to handle reads of varying lengths: a shorter sequence is considered identical to a longer one if it perfectly matches its 5′ prefix. If duplicates are found, the pair containing the longest reads is retained.

While `FastUniq` guarantees absolute precision for exact duplicate removal (zero mismatches allowed), its architectural design introduces critical resource bottlenecks. Because it relies on importing the entire dataset into RAM simultaneously using a hierarchical storage structure, its memory footprint scales linearly with the input file size. Benchmarking studies have highlighted this as a severe limitation for modern high-throughput data; for instance, processing a 538 GB Hi-C dataset required up to 1 TB of RAM [6].

Consequently, while highly accurate, `FastUniq` is often impractical for massive datasets without access to HPC clusters. Furthermore, its scope is strictly limited to paired-end data, it does not tolerate sequence mismatches, and its deduplication output is known to be sensitive to the order of the input files.

## 2.3 : Clumpify

`Clumpify`, a component of the Java-based BBTools suite [7], introduces a distinct algorithmic paradigm to duplicate removal based on $k$-mer grouping. Rather than relying on exact sequence hashing or global

coordinate sorting, `Clumpify` identifies reads that share common $k$-mers and groups them into localized clusters or "clumps".

This spatial reorganization provides a unique secondary benefit. By physically placing highly similar sequences adjacent to one another in the output file, `Clumpify` dramatically improves standard `gzip` compression efficiency, as the compression algorithm can utilize much closer reference pointers.

A major functional advantage of `Clumpify` is its high sensitivity to optical duplicates. By parsing the flowcell tile and pixel coordinates embedded within Illumina FASTQ headers, `Clumpify` (via parameters such as `dupedist`) can explicitly distinguish between amplification-derived PCR duplicates and sensor-derived optical duplicates. For paired-end data, `Clumpify` predominantly uses the first read (R1) to determine the clump, allowing the paired read (R2) to passively follow its counterpart, which maintains file synchronization while minimizing computational overhead.

To manage memory, `Clumpify` implements a multi-phase strategy (KmerSplit and KmerSort). If the dataset exceeds available RAM, the tool can divide the data into temporary groups on the disk, sorting them individually before merging.

However, benchmarking shows that its reliance on the Java Virtual Machine (JVM) can lead to variable stability and high memory overhead on massive datasets. Sometimes it requires multi-threading just to maintain stability on large inputs like 500+ GB Hi-C datasets [6].

## 2.4 : SeqKit

`SeqKit` [8,9] is a comprehensive, cross-platform, and ultra-fast toolkit for FASTA and FASTQ file manipulation. Written in the Go, it benefits from efficient concurrency management and the ability to produce static binaries with no external dependencies, making it highly accessible and stable across different operating systems.

For PCR deduplication, SeqKit provides the `rmdup` subcommand. This tool relies on an exact sequence matching approach using a hashing strategy. To process massive datasets without exhausting system memory, `seqkit rmdup` offers an MD5 mode (invoked via the `-m` or `--md5` flag). In this mode, the algorithm computes and stores only the MD5 digest of each sequence in the RAM rather than the full nucleotide sequence. This extreme memory efficiency allows SeqKit to maintain a very low resource footprint, even outperforming several newer tools when handling vast amounts of single-end (SE) data.

Despite its speed and memory efficiency, `SeqKit rmdup` has a notable architectural limitation: it does not natively support the synchronized deduplication of paired-end reads. The `rmdup` command evaluates and processes individual files independently. It will check duplicates inside each file separately, so if $R1_1 = R1_2$ but $R2_1 \neq R2_2$, $R1_2$ in will be flagged as a duplicate and removed, while $R2_2$ will be retained, leading to loss of data and desynchronization of the paired files. While workarounds exist using other SeqKit subcommands (such as `seqkit pair` to resynchronize files), `SeqKit rmdup` is generally considered the optimal choice strictly for single-end libraries.

## 2.5 : CD-HIT-DUP

`CD-HIT-DUP`, a specialized utility within the broader CD-HIT package [10,11], is designed to identify and remove both identical and nearly-identical duplicates. Unlike tools restricted to exact string matching, `CD-HIT-DUP` is particulary robust against sequencing errors. It incorporates a greedy incremental clustering algorithm that tolerates a user-defined threshold of mismatches, including both substitutions and insertions/deletions.

`CD-HIT-DUP` relies on a prefix-suffix comparison approach. The tool first bins reads that share an identical prefix of $k$ nucleotides. To accelerate this binning and sorting process, it uses a dual numeric encoding strategy (initially base-5, then base-10), allowing prefixes of up to 27 nucleotides to be compactly represented as 64-bit integers. Within each bin, sequences are sorted by decreasing length, with the longest sequence acting as the cluster's seed. The algorithm then evaluates the suffixes of the remaining sequences against this

seed, utilizing a $k$-mer word-based index table to statistically predict sequence identity and bypass computationally expensive full pairwise alignments when possible. For paired-end libraries, `CD-HIT-DUP` conceptually joins the read pairs into a single sequence and checks prefixes at both ends simultaneously to ensure paired-end integrity.

While its error tolerance provides a nuanced, high-sensitivity approach to duplicate removal, `CD-HIT-DUP` introduces a severe computational bottleneck regarding memory consumption. Because the greedy clustering algorithm must retain representative sequences and cluster bins in RAM, its memory footprint scales aggressively with dataset size and the allowed mismatch threshold. Benchmarking has shown that processing a relatively modest 50 million single-end read library can consume between 33 GB and 53 GB of RAM. For massive modern datasets, such as a 538 GB Hi-C dataset, `CD-HIT-DUP` required approximately 1 TB of RAM, often crashing on resource-constrained hardware [6].

Furthermore, practical deployment of `CD-HIT-DUP` in modern automated workflows requires caution. The tool has known stability issues in containerized environments (e.g., Docker and Singularity), occasionally throwing segmentation faults or I/O assertion errors when inputs are provided via process substitution from compressed files.

## 2.6 : PRINSEQ++

`PRINSEQ++` [12] is a highly optimized C++ implementation of the Perl-based `prinseq-lite` program. It is designed to perform a comprehensive suite of quality control, filtering, trimming, and reformatting tasks for genomic and metagenomic sequence data. By utilizing POSIX threads (pthreads) for parallelization, it achieves processing speeds more than 10 to 16 times faster than its predecessor while maintaining a relatively low memory footprint.

For sequence deduplication, `PRINSEQ++` departs from exact string comparison or sorting heuristics by implementing a probabilistic data structure known as a Bloom filter. During execution, each sequence is transformed using multiple fast, non-cryptographic hash functions, and the corresponding indices in a shared bit-array are flagged. If a newly evaluated sequence maps entirely to bits that are already set to positive, it is classified as a duplicate. This probabilistic model is highly advantageous for multi-threaded environments because it enables asynchronous read and write access to the filter with minimal thread blocking.

Additionally, `PRINSEQ++` natively supports the direct reading and writing of `gzip`-compressed FASTQ and FASTA files without the need for intermediate disk I/O, which significantly reduces hard drive storage requirements. It also natively supports paired-end reads by processing pairs simultaneously and outputting synchronized R1 and R2 files.

However, the deduplication function (`-derep`) is strictly limited to exact duplicates and does not tolerate sequencing errors. Furthermore, the probabilistic nature of the Bloom filter introduces a negligible but theoretically non-zero rate of false positives. Finally, when `PRINSEQ++` is executed with multiple threads, the output sequences are not guaranteed to remain in the same order as the input files, which may require consideration if downstream tools expect strict global index synchronization.

## 2.7 : SAM/BAM ecosystem tools

In addition to tools that operate directly on raw FASTQ sequences, the bioinformatics ecosystem relies heavily on utilities designed for SAM and BAM formats. PCR deduplication tools like such as `samtools` (via `markdup`) [13] and Picard's `MarkDuplicates` [14].

While these tools are typically used downstream after mapping reads to a reference genome, it is technically possible to convert raw, unaligned FASTQ files directly into unmapped SAM/BAM files to process them through these pipelines. However, even without the computationally heavy step of genomic alignment, the sheer format conversion from FASTQ to SAM/BAM introduces significant and unnecessary I/O and processing overhead. Because our primary objective is to evaluate and develop a highly optimized, memory-efficient tool strictly for direct FASTQ-to-FASTQ deduplication, these format-dependent tools fall outside the scope of our comparative analysis.

# 3 – FastDedup: High-Performance FASTX Deduplication

The goal behind the development of `FastDedup` (or `FDedup`) was to create a FASTX PCR deduplication tool that prioritizes maximum speed and memory efficiency. To achieve this, the tool relies on `xxh3` [15,16], a rapid non-cryptographic hash function, to compute a unique fingerprint for each single-end or paired-end read. These fingerprints are cached in memory using `fxhash` [17], which provides a low memory footprint.

FDedup provides users with precise control over the hash collision rate while maintaining high performance. It automatically scales its hashing strategy, seamlessly switching between 64-bit and 128-bit hashes based on the estimated input sequence count and a user-defined collision probability threshold.

Specifically designed for seamless integration into OMICS pipelines, the tool features robust incremental deduplication and auto-recovery mechanisms. In the event of an interruption FDedup can safely preload existing hashes from the output file to prevent duplicate processing and smoothly resume operations. If an uncompressed output file becomes corrupted due to a crash, the tool automatically detects the issue, calculates a fail-safe truncation point, and truncates the file to the last valid sequence before continuing.

Finally, a dry-run mode (`--dry-run` or `-s`) is available to calculate the duplication rate without writing any output files. Since file I/O operations account for the majority of execution time, this mode is exceptionally fast and serves as a highly efficient feature for pipeline planning and data analyses.

## 3.1 : Algorithmic Complexity and Hardware-Level Optimizations

The architectural principal of `FastDedup`'s performance lies in shifting the deduplication problem from a string-matching paradigm to an integer-matching paradigm. By transforming raw paired-end sequences (e.g., 2 x 150 bytes) into a single 64-bit hash, reducing the data by 37.5 times in both space and time complexity.

### 3.1.1 Space Complexity and Memory Footprint

In a deterministic approach, storing a read pair requires retaining sequences of length $L$ (where $L = r_1 + r_2$). The space complexity for $N$ unique sequences scales linearly as $O(N \times L)$. For a standard 150 bp paired-end dataset, each pair consumes at least 300 bytes of raw character data, plus substantial data structure overhead (e.g., string pointers). Storing $10^8$ sequences typically exceeds 32 GB of RAM. By computing a 64-bit hash using `xxh3`, `FastDedup` reduce the sequence identity into exactly 8 bytes. This decouples the memory requirement from the read length, reducing the space complexity to $O(N)$ (since $8 \ll L$).

### 3.1.2 Time Complexity and ALU Efficiency

String-based deduplication necessitates sequential, byte-by-byte comparisons. In the worst-case scenario (ex: highly similar sequences or long identical prefixes), comparing two sequences requires $O(L)$ time. Conversely, while calculating the initial hash takes $O(L)$ time during the single-pass file reading, all subsequent identity verifications are reduced to integer operations. Comparing two 64-bit integers is executed natively by the CPU's Arithmetic Logic Unit (ALU) in exactly 1 clock cycle, effectively reducing the hash lookup and comparison time complexity to $O(1)$.

### 3.1.3 Cache Locality and Zero-Allocation Pipeline

Beyond theoretical Big-$O$ improvements, the 8-byte hash architecture heavily exploits modern CPU hardware design. Processors fetch data from RAM in discrete 64-byte cache lines. A 300-byte sequence spans multiple cache lines, leading to frequent cache misses and severe memory bandwidth bottlenecks. In contrast, exactly eight 64-bit hashes fit perfectly into a single cache line. As `fxhash` probes the internal hash set for collisions, it operates almost exclusively within the ultra-fast CPU cache (L1/L2), minimizing latency and maximizing throughput.

Furthermore, `FastDedup` processes byte slices directly from the `needletail` [18] parser into the `xxh3` hasher without ever allocating the raw string to the heap. This zero-allocation pipeline completely bypasses system-level memory allocation (`malloc`) overheads, ensuring that the execution time is strictly bound by disk I/O rather than RAM latency.

## 3.2 : The Mathematics of Dynamic Hashing

While `FDedup` defaults to the highly efficient 64-bit hashing strategy of `xxh3`, it dynamically assesses the risk of hash collisions to guarantee data integrity. The theoretical risk of a hash collision $p$ for $x$ sequences follows the standard Birthday Problem approximation [19,20]:

$$p = \frac{x^2}{2 * 2^N}$$

Where $N \in \{64, 128\}$ represents the bit-length of the hash. At the default 64-bit configuration and a probability threshold of 0.001, it requires approximately $0.19 * 10^9$ sequences to reach a 1‰ collision risk. If the estimated dataset size pushes the collision probability beyond this threshold, `FDedup` automatically scales to 128-bit hashing, effectively nullifying the risk (requiring $0.28 * 10^{17}$ sequences to reach the same probability).

For paired-end reads, maintaining this theoretical probability requires bitwise operations. Let $S_1$ and $S_2$ represent the nucleotide sequences of Read 1 and Read 2, and $H$ be our non-cryptographic uniform hash function mapping to $\{0, 1\}^N$. We define the independent hashes for each read as:

$$h_1 = H(S_1)$$

$$h_2 = H(S_2)$$

If the algorithm combined these hashes using a simple bitwise XOR ($\oplus$), a critical failure occurs when Read 1 and Read 2 are identical sequences ($S_1 = S_2$). In this case, $h_1 = h_2$, resulting in:

$$h_c = h_1 \oplus h_1 = 0$$

Every paired-end read with identical forward and reverse sequences would map to 0. This would cause massive, deterministic hash collisions, meaning the collision probability would no longer be bounded by the theoretical uniform distribution, but rather by the biological frequency of identical sequence pairs in the dataset.

To prevent symmetrical cancellation, the algorithm introduces a bitwise left-rotation by $\frac{N}{2}$ bits, denoted as $R(x, k)$. The unified fingerprint $h_u$ is instead calculated as:

$$h_u = h_1 \oplus R\left(h_2, \frac{N}{2}\right)$$

Under this operation, even if $S_1 = S_2$:

$$h_u = h_1 \oplus R\left(h_1, \frac{N}{2}\right) \neq 0$$

Because $R\left(h_1, \frac{N}{2}\right)$ shifts the lower half of the bits to the upper half and vice versa, $h_1 \oplus R\left(h_1, \frac{N}{2}\right)$ will almost never equal 0. It would only equal 0 if $h_1$ consisted of two perfectly identical halves, an event with an astronomically low probability of $2^{-\frac{N}{2}}$.

Since a bitwise rotation is a bijective function over the $N$-bit vector space, it perfectly preserves the original entropy of $h_2$. When two independent, uniformly distributed random variables are combined using a bitwise XOR, the resulting variable remains uniformly distributed. Therefore, the unified fingerprint $h_u$ safely maintains a strict uniform distribution, ensuring the overall collision risk remains uncorrupted and strictly bounded by the initial probability formula.

This is the mathematical foundation that allows `FDedup` to guarantee a collision probability of less than 1‰ for datasets containing up to approximately 190 million paired-end reads (380 million total sequences) when using 64-bit hashes.

## 3.3 : Architecture

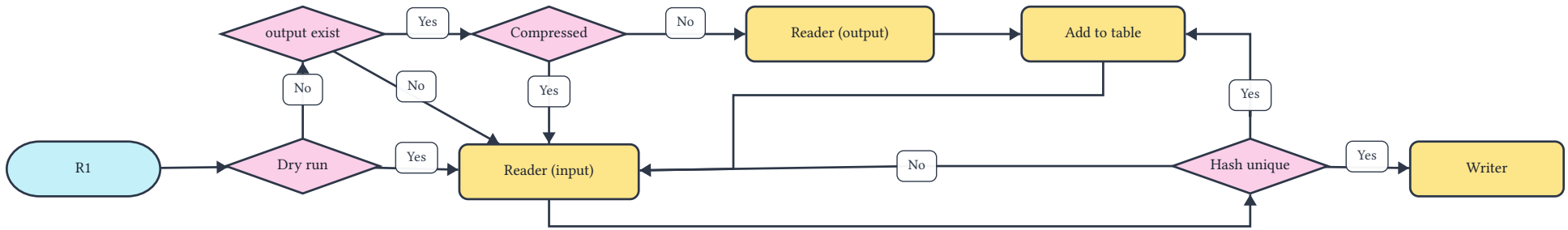


Figure 2: Architectural design of `FastDedup` for Single-End (SE) reads.

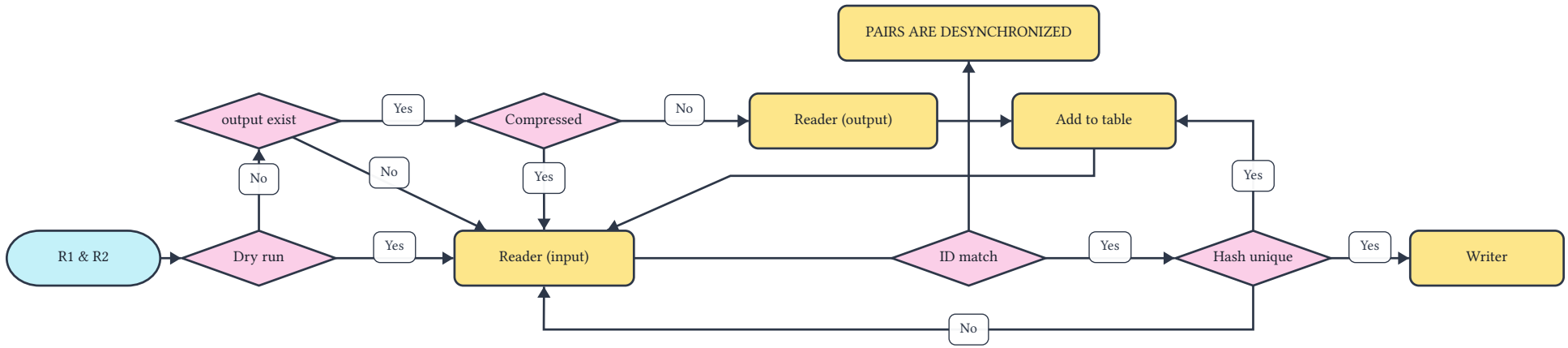


Figure 3: Architectural design of `FastDedup` for Paired-End (PE) reads.

# 4 Benchmarking and performance evaluation

Setup that it OOM at 32GB of RAM usage because who would, in its right mind, put 32GB of RAM on a deduplication tool? It's also single core because it's useless to parallelize a tool that is already faster than the disk I/O bottleneck. (Faudra le dire mieux ça)

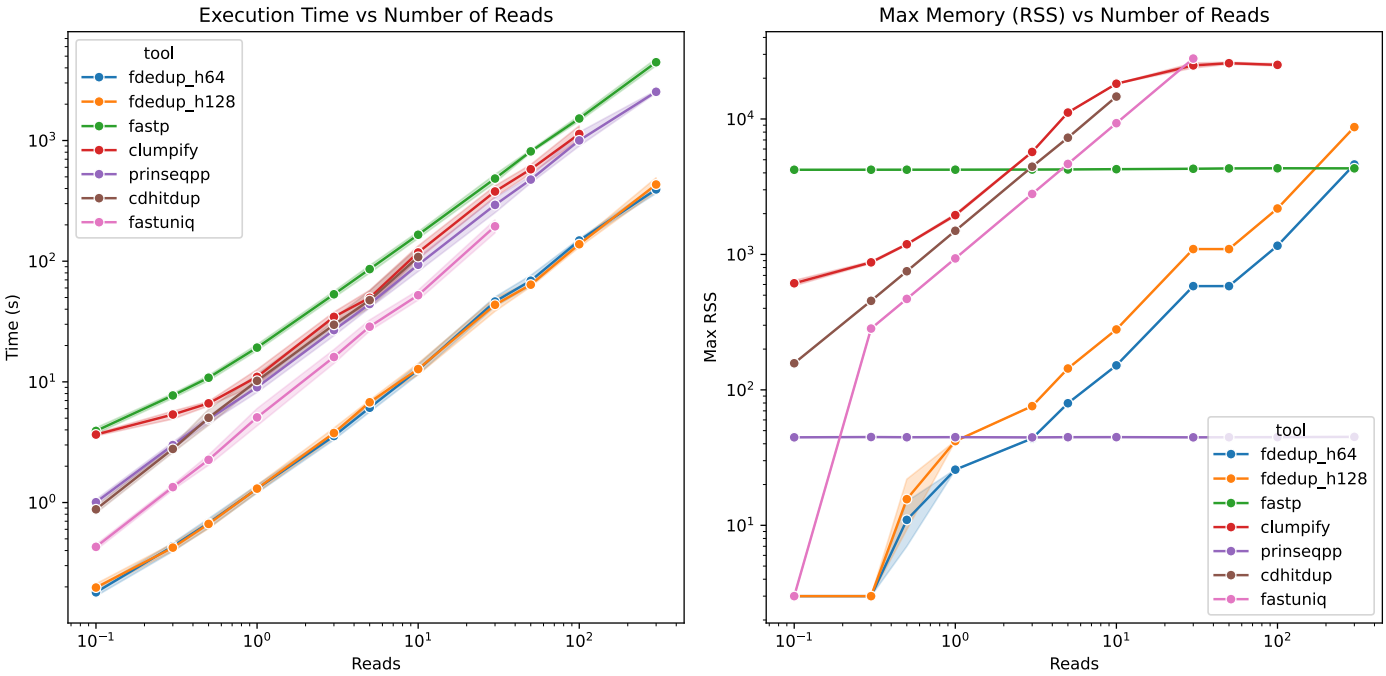


Figure 4: Execution time and memory usage of `FastDedup` compared to existing tools across varying dataset sizes.

| Tool | Algorithm / Approach | PE Support | Error Tolerance | Memory Usage | Key Limitations |
|---|---|---|---|---|---|
| `fastp` [3,4] | Probabilistic (Bloom filter/hashes) | Yes | No | Configurable | False-positive rate; long runtime |
| `FastUniq` [5] | Deterministic (Merge sort) | Yes | No | Very High | Severe RAM bottleneck; sensitive to input file order. |
| `Clumpify` [7] | k-mer grouping | Yes | Yes | Variable | Variable stability on massive datasets; high memory overhead. |
| `SeqKit rmdup` [8,9] | Exact matching (Hashing/MD5) | No | No | Very Low | Desynchronizes paired-end files. |
| `CD-HIT-DUP` [10,11] | Greedy incremental clustering | Yes | Yes | Very High | Stability issues in containerized environments. |
| `PRINSEQ++` [12] | Probabilistic (Bloom filter) | Yes | No | Low | False-positives rate |
| `FastDedup` | Integer-matching (Dynamic hashing) | Yes | No | Very Low | Theoretical negligible collision risk |

Table 1: Summary of existing tools for reads deduplication and their limitations.

| Tool | 0.1M | 0.3M | 0.5M | 1.0M | 3.0M | 5.0M | 10.0M | 30.0M | 50.0M | 100.0M | 300.0M |
|---|---|---|---|---|---|---|---|---|---|---|---|
| cdhitdup | 0.88s<br>157.2MB | 2.8s<br>454.2MB | 5.0s<br>751.3MB | 10.2s<br>1494.9MB | 29.7s<br>4444.0MB | 47.5s<br>7274.6MB | 108.4s<br>14639.0MB | OOM | OOM | OOM | OOM |
| clumpify | 3.7s<br>612.4MB | 5.4s<br>873.8MB | 6.6s<br>1188.1MB | 11.0s<br>1951.0MB | 34.6s<br>5709.9MB | 49.7s<br>11151.7MB | 118.1s<br>18208.1MB | 378.3s<br>24856.8MB | 578.0s<br>25812.7MB | 1133.2s<br>25088.4MB | OOM |
| fastp | 3.9s<br>4217.0MB | 7.7s<br>4220.3MB | 10.8s<br>4221.3MB | 19.2s<br>4222.7MB | 53.2s<br>4231.6MB | 86.0s<br>4239.4MB | 165.6s<br>4260.2MB | 484.1s<br>4289.1MB | 812.7s<br>4314.4MB | 1521.3s<br>4325.8MB | 4453.5s<br>4319.7MB |
| fastuniq | 0.43s<br>3.0MB | 1.3s<br>283.5MB | 2.3s<br>469.5MB | 5.1s<br>934.5MB | 16.1s<br>2796.0MB | 28.7s<br>4657.5MB | 52.3s<br>9312.0MB | 194.5s<br>27927.0MB | OOM | OOM | OOM |
| fdedup_h128 | 0.20s<br>3.0MB | 0.42s<br>3.0MB | 0.66s<br>15.6MB | 1.3s<br>41.9MB | 3.8s<br>75.9MB | 6.8s<br>143.8MB | 12.8s<br>279.4MB | 43.5s<br>1095.4MB | 63.9s<br>1095.0MB | 138.3s<br>2183.7MB | 432.0s<br>8711.6MB |
| fdedup_h64 | 0.18s<br>3.0MB | 0.43s<br>3.0MB | 0.68s<br>11.0MB | 1.3s<br>25.8MB | 3.6s<br>43.8MB | 6.1s<br>79.7MB | 12.6s<br>151.6MB | 46.3s<br>583.5MB | 68.9s<br>583.6MB | 148.4s<br>1159.4MB | 393.4s<br>4615.6MB |
| prinseqpp | 1.0s<br>44.6MB | 3.0s<br>44.9MB | 4.9s<br>44.7MB | 9.0s<br>44.8MB | 26.8s<br>44.6MB | 44.2s<br>44.7MB | 93.4s<br>44.8MB | 292.7s<br>44.6MB | 474.6s<br>44.7MB | 1002.2s<br>44.8MB | 2533.5s<br>45.0MB |

Table 2: Runtime and memory usage benchmarking of existing tools and `FastDedup` on synthetic datasets of varying sizes. OOM indicates Out Of Memory errors.

# 5 Discussion and future perspectives

# 6 Conclusion

# 7 Acknowledgements

Code architecture and design was developed by Raphaël Ribes, assisted by Gemini 3.1. Feedback on the algorithm design and ideas for features were provided by Céline Mandier, and the implementation was done by Raphaël Ribes. The tests suite was mostly developped by Gemini 3.1 and Claude Sonnet 4.6, reviewed by Raphaël Ribes.

This paper was written by Raphaël Ribes, reviewed and edited by Céline Mandier, with the assistance of Gemini 3.1 for grammar, and language correction.

# Bibliography

[1] M. T. W. Ebbert, M. E. Wadsworth, L. A. Staley, K. L. Hoyt, B. Pickett, J. Miller, J. Duce, J. S. K. Kauwe, and P. G. Ridge, Evaluating the necessity of PCR duplicate removal from next-generation sequencing data and a comparison of approaches, BMC Bioinformatics **17**, 239 (2016).

[2] Z. Zhang, L. Zhang, G. Zhang, Z. Zhao, H. Wang, and F. Ju, Deduplication Improves Cost-Efficiency and Yields of De Novo Assembly and Binning of Shotgun Metagenomes in Microbiome Research, Spectrum **11**, e4282 (2023).

[3] S. Chen, Y. Zhou, Y. Chen, and J. Gu, fastp: an ultra-fast all-in-one FASTQ preprocessor, Bioinformatics **34**, i884 (2018).

[4] S. Chen, Ultrafast one-pass FASTQ data preprocessing, quality control, and deduplication using fastp, Imeta **2**, e107 (2023).

[5] H. Xu, X. Luo, J. Qian, X. Pang, J. Song, G. Qian, J. Chen, and S. Chen, FastUniq: A Fast De Novo Duplicates Removal Tool for Paired Short Reads, Plos ONE **7**, e52249 (2012).

[6] A. I. Sigorskikh, M. A. Kompaniets, I. S. Ilnitskiy, G. K. Ryabykh, and A. A. Mironov, Fastq-dupaway: a fast and memory-efficient tool for deduplication of single- and paired-end NGS data, Scientific Reports **15**, 45303 (2025).

[7] B. Bushnell, *Bbmap: A Fast, Accurate, Splice-Aware Aligner (And Bbtools)*, https://archive.jgi.doe.gov/data-and-tools/software-tools/bbtools/.

[8] W. Shen, S. Le, Y. Li, and F. Hu, SeqKit: A Cross-Platform and Ultrafast Toolkit for FASTA/Q File Manipulation, Plos ONE **11**, e163962 (2016).

[9] W. Shen, B. Sipos, and L. Zhao, SeqKit2: A Swiss Army Knife for Sequence and Alignment Processing, Imeta **3**, e191 (2024).

[10] L. Fu, B. Niu, Z. Zhu, S. Wu, and W. Li, CD-HIT: accelerated for clustering the next-generation sequencing data, Bioinformatics **28**, 3150 (2012).

[11] Y. Huang, B. Niu, Y. Gao, L. Fu, and W. Li, CD-HIT Suite: a web server for clustering and comparing biological sequences, Bioinformatics **26**, 680 (2010).

[12] V. A. Cantu, J. Sadural, and R. Edwards, PRINSEQ++, a multi-threaded tool for fast and efficient quality control and preprocessing of sequencing datasets, Peerj Preprints **7**, e27553v1 (2019).

[13] H. Li, B. Handsaker, A. Wysoker, T. Fennell, J. Ruan, N. Homer, G. Marth, G. Abecasis, and R. Durbin, The sequence alignment/map format and SAMtools, Bioinformatics **25**, 2078 (2009).

[14] Broad Institute, *Picard Toolkit*, http://broadinstitute.github.io/picard/.

[15] Y. Collet, *Xxhash: Extremely Fast Non-Cryptographic Hash Algorithm*, https://github.com/Cyan4973/xxHash.

[16] DoumanAsh, *Xxhash-Rust: Rust Implementation of Xxhash*, https://github.com/DoumanAsh/xxhash-rust.

[17] cbreeden, *Fxhash: A Fast, Non-Secure, Hashing Algorithm Derived from an Internal Hasher in Firefox*, https://github.com/cbreeden/fxhash.

[18] M. Gore, *Needletail: Fast FASTX Parsing and K-Mer Methods in Rust*, https://github.com/onecodex/needletail.

[19] W. Feller, *An Introduction to Probability Theory and Its Applications*, 3rd ed., Vol. 1 (John Wiley & Sons, 1968).

[20] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. (The MIT Press, 2009).